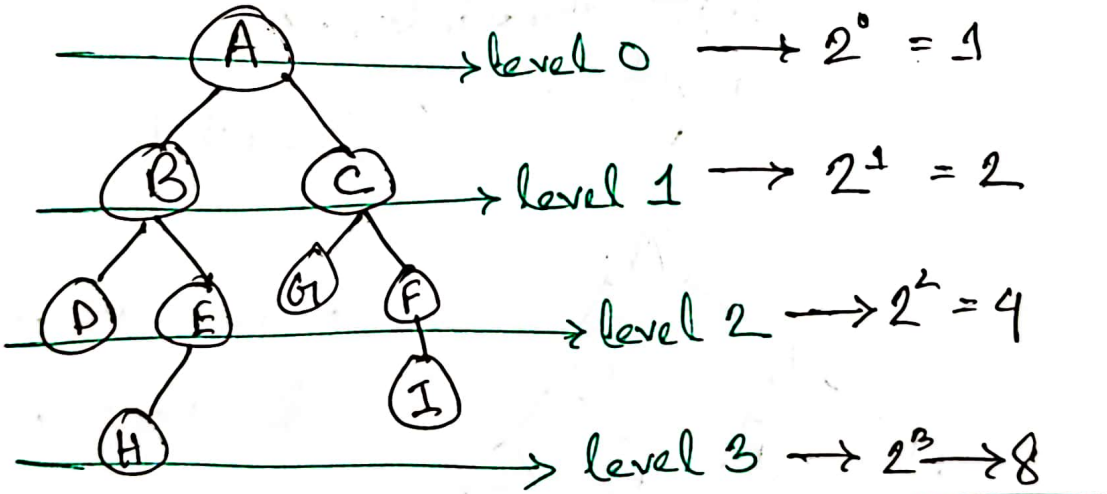
(*) print sum all the nodes in a Binary tree:

def print_sum (root):

Binary TREE

16-08-23

Array - Based representation ⟶ static representation



Level d:

total # of nodes $= 2^0 + 2^{+1} + 2^2 + \ldots + 2^d$

$= \boxed{2^{d+1} - 1}$

Emistat®
Ondansetron

CamScanner

Given an Array Based representation of Binary Tree:

draw the tree

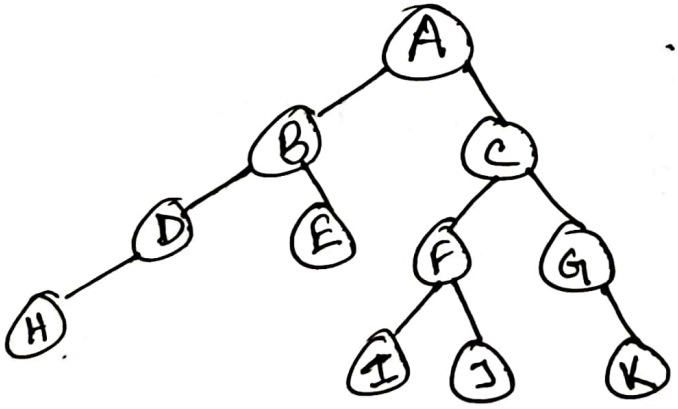
array_rep = [None, A, B, C, D, E, F, G, H, None, None, None, I, J, None, K]
(indices 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15)

1) Root always at index = 1

2) parent_index = i

conditions: left child index = 2i

right " " = 2i + 1



No need to draw the None nodes.

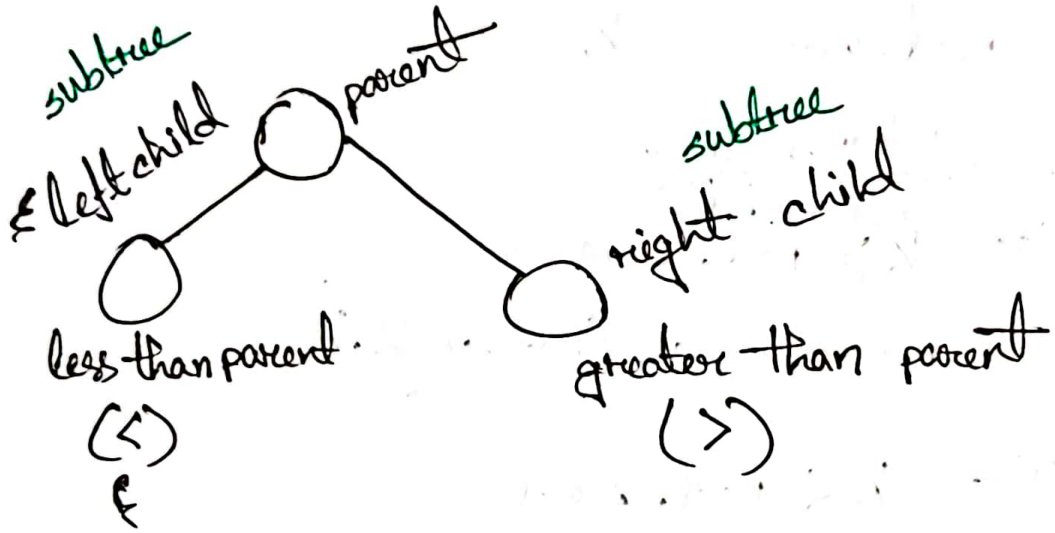
∴ Higest num ⟶ $2^{3+1} - 1$

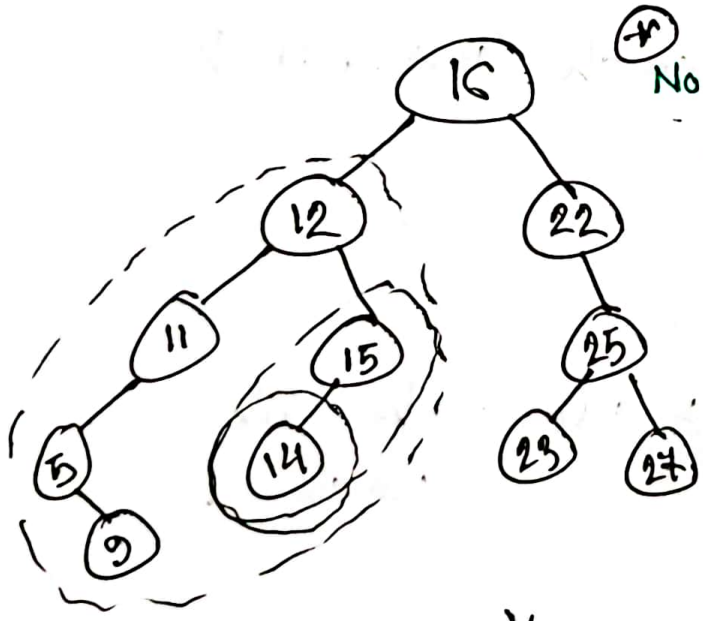
$= 2^4 - 1 = 15$

Array size ⟶ 16

↳ Because of None at idx 0.

Binary Search Tree (BST)





* No value can be greater than 16 → on the left.

* on the right
↳ values has to be greater than 22.

* search for 14 in the BST:
↳ search in the left → 1st.

Healthcare
Emistat®
Ondansetron
CamScanner

code:

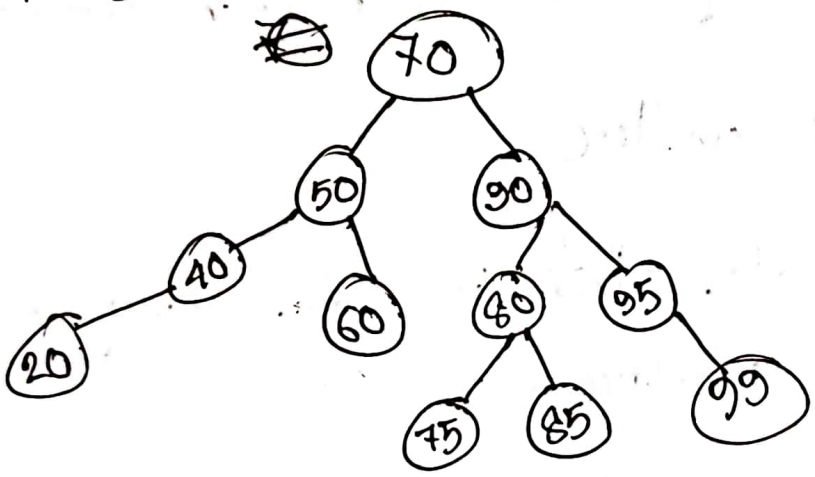
```
def search (root, k):
    if root.elem == k:
        return True
    if root == None:
        return False
    elif k > root.elem:
        return search (root.right, k)
    elif k < root.elem
        return search (root.left, k)
```

(#) Insert (4) / (11.64)
↳ code works like search.

(#) insert (15)
↳ Duplicate value can't be present in BST.

(4) Insert the following nodes in the BST.

70, 50, 40, 90, 20, 60, 20, 95, 99, 80, 85, 75



(5) Delete a node from BST:

1) 0 child → delete 20
2) 1 child → delete 95
3) 2 child → replace 2 children node with a node in the

1) Node (20) → None left sub-tree.
2)

CamScanner

(2) replace 2 children node with a node in left sub-tree.

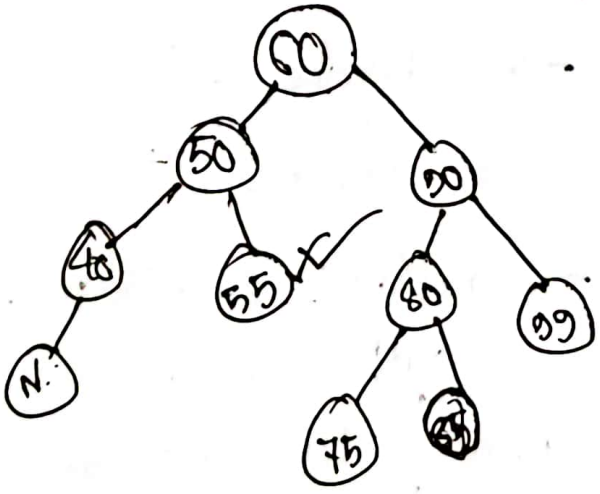
1) right greater → must ✓ fulfilled

2) left smaller

pick a node that is the largest in left sub-tree.

↳ right-most node in left sub-tree.

↳ inorder predecessor



in-order

40 – 50 – 55 – 60 – 75 – 80 – 85 – 90 – 99

actual tree

40 – 50 – 55 – 60 – 70 – 75 – 80 – 85 – 90 – 99

inorder successor

in right subtree

1) left smaller ✓ → must
2) right greater

→ pick a node that is the smallest in right sub-tree
↳ leftmost right node in the right subtree.

Emistat®

CamScanner

(#) Find minimum value in a BST:

```
def minimum (root):
    ~~if root.z == None:~~
    if root.left == None:
        return root.elem
    return minimum (root.left)
```

CamScanner

find height of a tree :

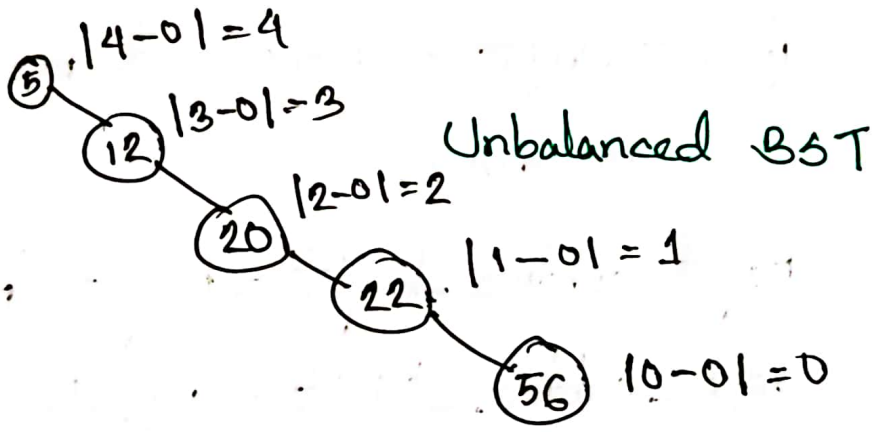
```
def height (root):
    if root.left == None and root.right == None:
        return 0.
    right_subtree_height = height (root.right)
    left_subtree_height = height (root.left)
    m = max (right... , left...)
    return m+1
```

Emistat®

CamScanner

Balanced BST:

a = [5 12 20 22 56]



| right subtree height − left subtree height | > 1

